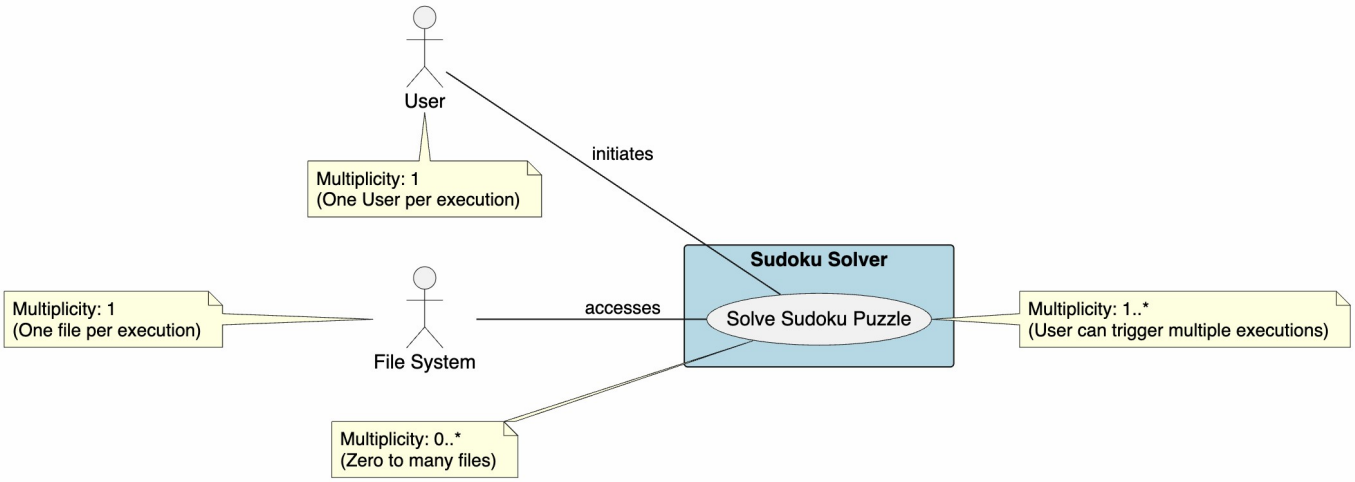


UML diagram created on PlantUML

a. Identify/discover a use case

- Use case: Sudoku solver puzzle

b. Provide a brief description for the use case

- This use case allows the User to run the Sudoku Solver, view each original puzzle as it was read from its file, and see every solution the program finds together with the total number of solutions. If the User does not name any puzzle files, the use case operates on the default files puzzle1.txt through puzzle5.txt.

c. Provide a step-by-step flow (outline) of the events for the use case

1. User: starts the program, optionally supplying one or more puzzle filenames.
2. System: determines which files to process — the files named by the User, or puzzle1.txt through puzzle5.txt if none were named.
3. System: opens the current puzzle file and reads its 81 cells.
4. System: displays the original puzzle.
5. System: validates the starting numbers by checking for duplicates.
6. System: searches for all solutions to the puzzle.
7. System: displays each solution found and the total number of solutions.
8. System: advances to the next file and repeats steps 3–7 until every file has been processed.
9. System: terminates.

d. Provide a detailed description for the use case

- Main (basic) flow
  1. The User starts the program, optionally naming one or more puzzle files.
  2. The system selects the files to process: the named files, or the five default files.
  3. For the current file, the system opens the file and reads its 81 cells.
  4. The system displays the original puzzle.
  5. The system checks the starting numbers for duplicates.
  6. The system searches for all solutions to the puzzle.
  7. The system displays each solution and the total number of solutions found.
  8. The system repeats steps 3–7 for each remaining file.
  9. The system terminates normally.

- Alternate flows
  - A1 — File cannot be opened or read. At step 3, if the file is missing or unreadable, the system prints an error message for that file, skips it, and continues with the next file (step 8). The overall run still completes.
  - A2 — Invalid file format. At step 3, if the contents do not form a valid 81-cell grid, the system prints an error message for that file and continues with the next file.
  - A3 — Duplicate starting numbers. At step 5, if two identical digits appear in the same row, column, or 3×3 box, the system prints "Invalid starting puzzle." and does not attempt to solve that file, then continues with the next file.
  - A4 — Valid puzzle with no solution. At step 6, if the search finishes without finding a solution, the system prints "No solution found." and continues with the next file.
  - A5 — No filenames supplied. At step 2, if the User gave no arguments, the system substitutes the default files puzzle1.txt through puzzle5.txt and proceeds with step 3.
  - A6 — Multiple solutions. At step 7, if the search produced more than one solution, the system displays every solution and reports the correct total rather than stopping at the first one.

- Postcondition (success guarantee): For every file that was requested or defaulted to, the system has either displayed all solutions with the total count, or displayed a specific error message identifying the problem. The program has advanced past every file and terminated normally; no file is silently skipped without a message.

- Minimal guarantee: Even when every file fails validation or cannot be read, the program reports a message for each file and exits without crashing.